



Application Note

Phase-Accurate Synchronization of Multiple Vantage Systems Using the Multi-System Synchronization Module

Peter Kaczkowski and Yao-Sheng Tung
Verasonics Inc.

June 2017

Summary

This document describes the use of the Synchronization Module hardware and software configuration needed to share a single 250 MHz system clock between multiple Vantage systems, and thus enable synchronous and phase-aligned operation of multiple hardware sequencers. This arrangement permits coherent acquisitions with up to 2048 channels (using up to 8 Vantage 256 systems; it is also possible to synchronize different Vantage models). Each Vantage unit operates with its own host controller as when running by itself, and the data collected by each system can be processed as is normally done for real time display. The data can also be transferred to a common computer during the sequence or after the experiment, and processed coherently using the complete data set.

Table of Contents

Summary	2
Table of Contents	2
1. Introduction	3
2. Procedure Summary	4
2.1 Setup the Hardware	4
2.2 Setup the Software	4
2.3 Verify Synchronous Operation and Align Phase	4
3. Theory of Operation	5
4. System Wiring Diagram	7
4.1 System I/O Panel	7
4.2 Multiple System Synchronization Module Wiring	7
5. Script Programming for Sequence Synchronization	9
6. Example Script	11
6.1 Additional Scripting Notes	11
7. Verification and Phase Alignment	12
8. Real-time data transfer through TCP/IP	12
8.1 High Speed Ethernet Network Adapter and system configuration	13
8.1.1 Set up IP address	13
8.1.2 Set up a shared folder	13
8.2 Script Programming for real-time data transfer/combo	14
9. APPENDIX — Example script source code: SetUpL11_5vFlash_MultiSysSync.m	14

1. Introduction

Many ultrasound experiments require synchronization with external instruments such as lasers, motion stages in field mapping or robotically guided procedures, and synchronization with cardiac or respiratory phases. For these applications, the Verasonics system provides trigger input and output capability that is very straightforward to program in an acquisition sequence. Triggers provide timing synchronization accuracy on the order of 4 ns for output triggers, and four digital sample clock periods for the input triggers. This accuracy is more than sufficient for many applications, but when combining data from several Vantage systems connected to a large array, phase accurate synchronization between systems is required.

This document describes the hardware and software configuration needed to share a single 250 MHz system clock between multiple Vantage systems and thus enable synchronous and phase-aligned operation of multiple hardware sequencers. This configuration permits coherent acquisitions with up to 2048 channels (using up to 8 Vantage 256 systems; it is also possible to synchronize different Vantage models). Each Vantage unit operates with its host controller, and the data collected by each system can be processed as normally done, for real time display. The data can also be transferred to a common computer during the sequence or after the experiment, and processed coherently using the complete data set.

This Application Note describes how the hardware must be connected to provide proper signal integrity, and how the software sequences must be arranged to assure proper operation. The very simple L11-5v plane wave imaging example script “SetupL11_5vFlash.m” is used to illustrate how an acquisition sequence should be modified to enable synchronous operation, and the complete example script “SetupL11_5vFlash_MultiSysSync.m” is provided for configuration and testing. Finally, an oscilloscope-based test procedure is described to help the user verify that synchronization is working properly.

The Vantage system also permits synchronizing no more than two systems using clock and trigger cables. The cable-only configuration has the advantage of low hardware expense at the cost of some programming complexity and clock skew that must be adjusted. The associated Application Note “Phase-Accurate Synchronization Of Two Vantage Systems Using The External Clock And Trigger Interfaces” provides all necessary details, and is included with the Verasonics software in the Example Scripts.

2. Procedure Summary

2.1 Setup the Hardware

1. Set up the Vantage systems and their computers next to each other. Be sure to leave a little space between systems for unimpeded air flow.
2. Connect HDMI cables to the Sync Box as indicated in the wiring diagram, selecting one system as the MASTER. The MASTER provides the Clock Out signal to the Sync Box.
3. Power on systems in any order, and launch Matlab on all computers.

2.2 Setup the Software

4. Install the provided SetUpL11_5vFlash_MultiSysSync.m example script on all computers.
5. On the MASTER, edit the script to properly identify the system as the MASTER (set “master = 1”).
6. On the SLAVE systems, no change to the script is required because the flag is set to “slave” by default (“master = 0”).
7. On the SLAVE systems, run the script without a transducer plugged in, and continue at the no transducer query so that the windows appear and the system enters the wait state. (Note that VSX is launched automatically when running the script)
8. Within 10 seconds, run the script on the MASTER (no transducer). If the SLAVE sequence DMA operation times out (waiting for data) and quits (script is set to wait up to 10 seconds), re-launch the SLAVE scripts so that they are ready before the MASTER.
9. Freeze or Quit the scripts using the MASTER controls. The SLAVE will not acquire additional frames without the MASTER’s trigger, and thus the RcvData buffer will contain frames synchronous with those on the MASTER.

2.3 Verify Synchronous Operation and Align Phase

10. Connect oscilloscope probes (one to each system) to transducer connector outputs on the same connector pin (select an active channel!!!) and the ground clips to the shield spring. Launch the scripts as above.
11. Trigger the scope using the MASTER signal. It is helpful to set the high voltage slider on both systems above 10 V (11.3 V is obtained with a single click), and the trigger level to around 1 Volt.
12. Observe that the transmitter signals are not jittering or drifting with respect to each other. Measure the time delay between waveforms, if any. All systems should be synchronized to within ± 2 ns, and are usually much closer than that.

3. Theory of Operation

In a clock-synchronized configuration, the 250 MHz system clock is provided by one of the systems, designated the “Master”, via an HDMI cable plugged into the Clock OUT port on the rear panel. This clock is buffered and redistributed to all systems by the Sync Module. When a Vantage hardware unit is initialized (e.g. by running the “Version” or “VSX” command from the Matlab command line), it uses the externally provided 250 MHz clock if it is present; otherwise, it will use its own internal system clock (the internal system clock is always running). Thus the systems can be powered up in any order, but the HDMI cables must be installed prior to initializing the systems (initialization occurs when running VSX, Version, etc.). Since all systems use the same phase-aligned clock, all sequencers run in phase.

The host controllers run independent software sequences, and these are event-aligned using the trigger interfaces: in every frame, all sequences enter a wait for trigger state, and then the Master system emits a trigger pulse to launch all systems at the same instant. (The trigger pulse is also buffered and redistributed by the Sync Module). After one frame of acquisitions, the systems enter the wait state again, and are re-triggered when the Master is ready. The frame rate is controlled by the Master sequence, and the Slave systems simply have to have completed their frame acquisition, data transfer, and processing before the Master trigger occurs. Note that the trigger signals are carried by the HDMI cables, and the multi-system triggers are not explicitly programmed using trigger Sequence Control commands. All synchronization signals are generated by the hardware sequencer as needed to time align the systems using only the ‘MultiSysSync’ command.

For imaging applications, for example with $2 \times 256 = 512$ channels connected to a 512-element array, the two acquisition sequences are nearly identical, with a small number of differences in sequence timing between Master and Slave code. For example, the Master may use Time To Next Acquisition (TTNA) Sequence controls to set the frame rate, allowing enough time to complete all Transmit/Receive and data transfer (DMA) hardware operations comprising an acquisition frame, and with some spare time to allow for responding to an occasional GUI control action. The Slave might accomplish all of the same actions, but should not use a TTNA constraint to allow the hardware sequence to progress as fast as it can. Completely different sequences are permissible.

The fundamental requirement for sequence synchronization is that all Slave sequences must have completed the acquisition frame by the time the Master is ready to acquire its next frame. This includes any external processing functions and HW/SW sync points. If not ready, the Slave will miss a frame or more and would resynchronize with the next one; the Sync Module will not permit this and will stop the sequence with an error. This is done to prevent respective RcvData buffers from containing misaligned data. Multiple DMA transfers within a frame may also result in misalignment, and proper verification of timing is required when performing several separate ‘transferToHost’ events within a frame.

Combining data acquired by multiple systems can be done during sequence execution via the external function processing mechanism, or after sequence completion. Data may be transferred over a local network to one of the host computers, a third computer, or a network attached storage device. If data is to be transferred to nonvolatile storage or over a network during the sequence, it is important to consider both the typical and extreme times required for such transfers. If the transfer time is highly variable, an additional handshake mechanism may be required to ensure that the Master waits until the

Slaves are ready before triggering. Currently, hardware and software for gathering and post-processing the entire acquired data set is not provided by Verasonics, and is the responsibility of the user. Assistance in developing solutions is available.

4. System Wiring Diagram

4.1 System I/O Panel

An external interface I/O panel on the back of the Vantage system unit contains Clock Input and Output ports using HDMI format connectors, and input and output trigger BNC connectors. The Multi-System Sync mode does not require use of the BNC trigger ports.

NOTE: Do not connect the CLOCK ports to a standard HDMI video device!!!

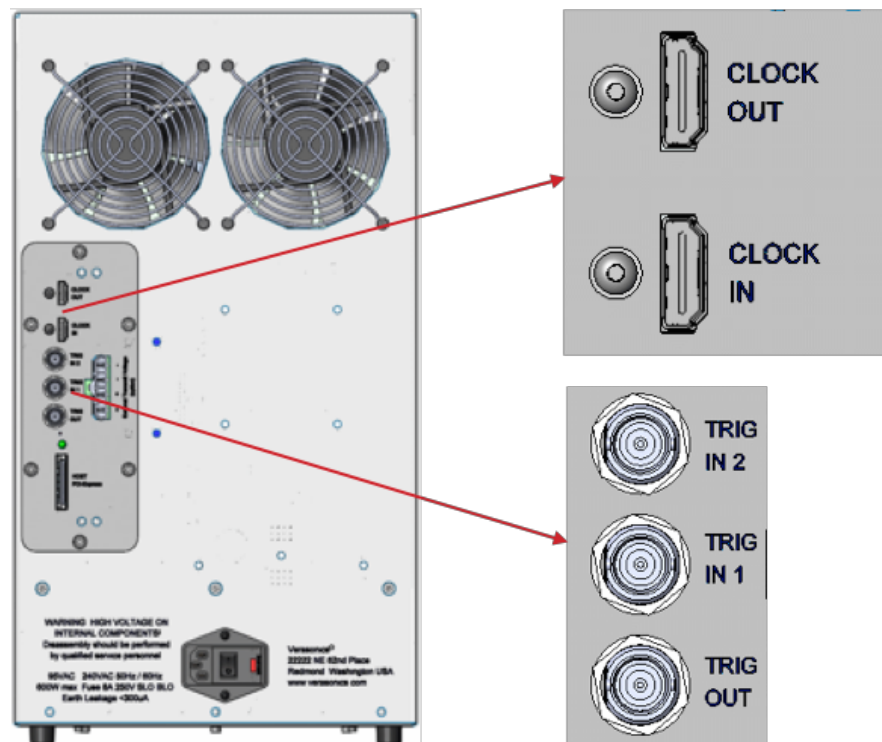


Figure 1. Rear panel connectors on Vantage system. The Synchronization Module provides its own trigger and synchronization signals, included in the cable connected to the HDMI ports. The user can program the Trigger signals (BNC connectors) independently, for use with the Master system.

4.2 Multiple System Synchronization Module Wiring

Hardware synchronization is achieved by connecting the master and slave systems to the Multi-System Synchronization Module with HDMI cables as shown in Fig. 2, and initializing the Slaves *after* the Master is powered on with the HDMI cables plugged in. This sequencing ensures that the externally provided clock signal is already present on the CLOCK IN ports when the Slave sequencers initialize. Otherwise, the Slaves will use their internal clocks, and the system sequences may appear to run normally but data frames will not be synchronized.

The Master sequence initiates the trigger when the Sequence Control condition is ‘normal’. The systems can also be triggered using the External TRIG IN port on the Sync Module. To activate this mode the condition must be either ‘BNC_Rising’ or ‘BNC_Falling’, depending on the desired active external trigger transition edge.

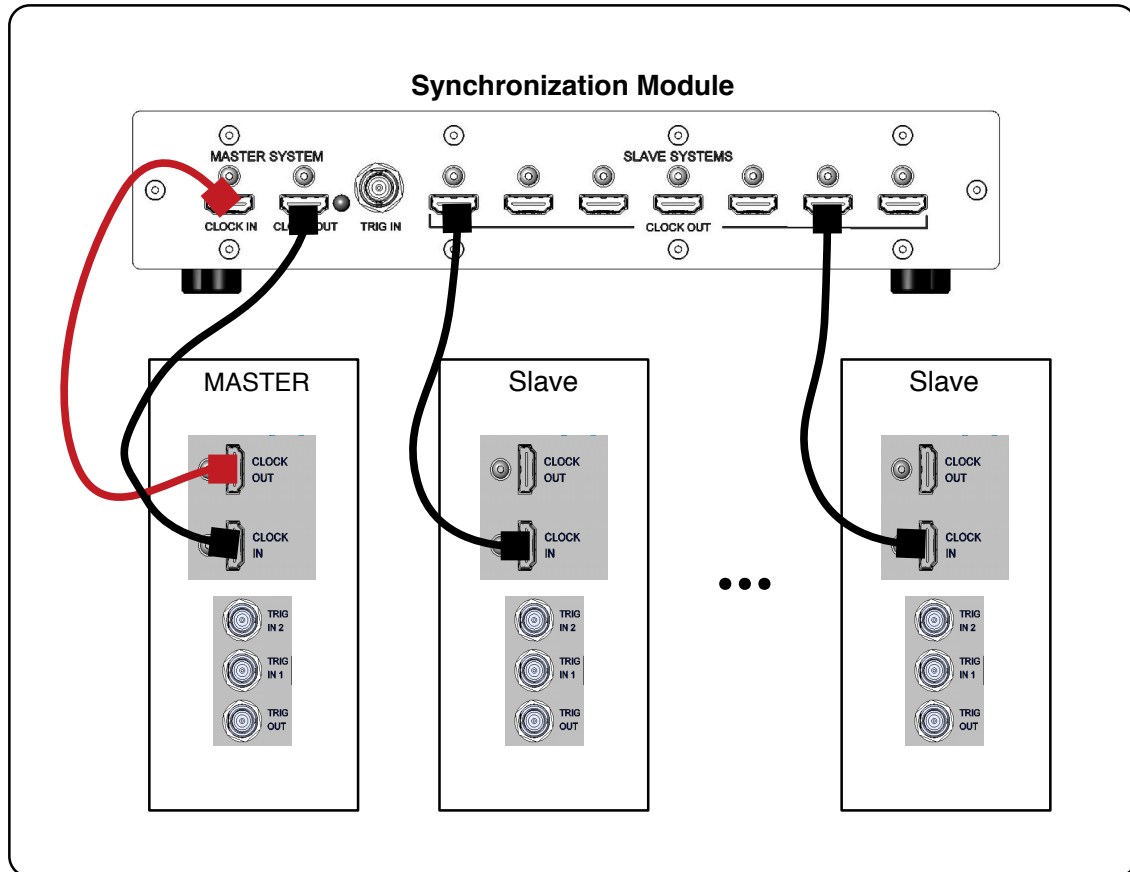


Figure 2. Wiring Diagram. The (black) cables connected to the system Clock IN ports should all be the same length to avoid phase shift between systems (preferably, all cables should be identical). Power is supplied to the Synchronization Module via the (red) cable from the Master. The trigger pulses are also sent over the HDMI cables.

5. Script Programming for Sequence Synchronization

The Master system clock provides the 250 MHz clock signal for the slave system for phase accuracy. The individual systems must also align their sequences to the same event, and the 'MultiSysSync' synchronization sequence control command is used to accomplish this alignment. In the simplest configuration, both systems run the same (almost) sequence program, which contains a few important modifications to a conventional script, as described below.

After initial setup events and upon entering the event loop, the synchronization Sequence Control pauses the system, and waits for a trigger input from the Sync Module. This trigger input either comes from the Master sequence, or from an external trigger source that connects to the Sync module via a BNC coax connector. This external trigger input facilitates triggering the acquisitions from an independent device that is operating asynchronously with respect to the Verasonics systems, such as an ECG trigger or a mechanical positioning system.

If systems are executing different sequence types, it is important to note that all systems must receive data at the same ADC frequency. In software versions 2.x, the ADC frequency is set by the choice of `Trans.frequency` and the `Receive.samplesPerWave` parameter. It is also essential that all systems be able to complete the frame before the start trigger for the next acquisition is issued.

Below is a flow chart describing the approach to software synchronization using a simple imaging script that loops over an acquisition, data transfer, and processing events. In RED are the new items required to convert a regular script to one that permits multi-system synchronization using HDMI cables and the Sync Module.

Major components of a sequence program are identified in the flowchart, but each "box" does not imply a single event definition. The 'MultiSysSync' Sequence Control command should be included in the same event as the first transmit/receive acquisition event to minimize delay (there is an overhead of about 2.5 microseconds between events). Other modifications to a conventional script are identified in red, and these represent the specific changes made to the Flash imaging script provided with this App Note.

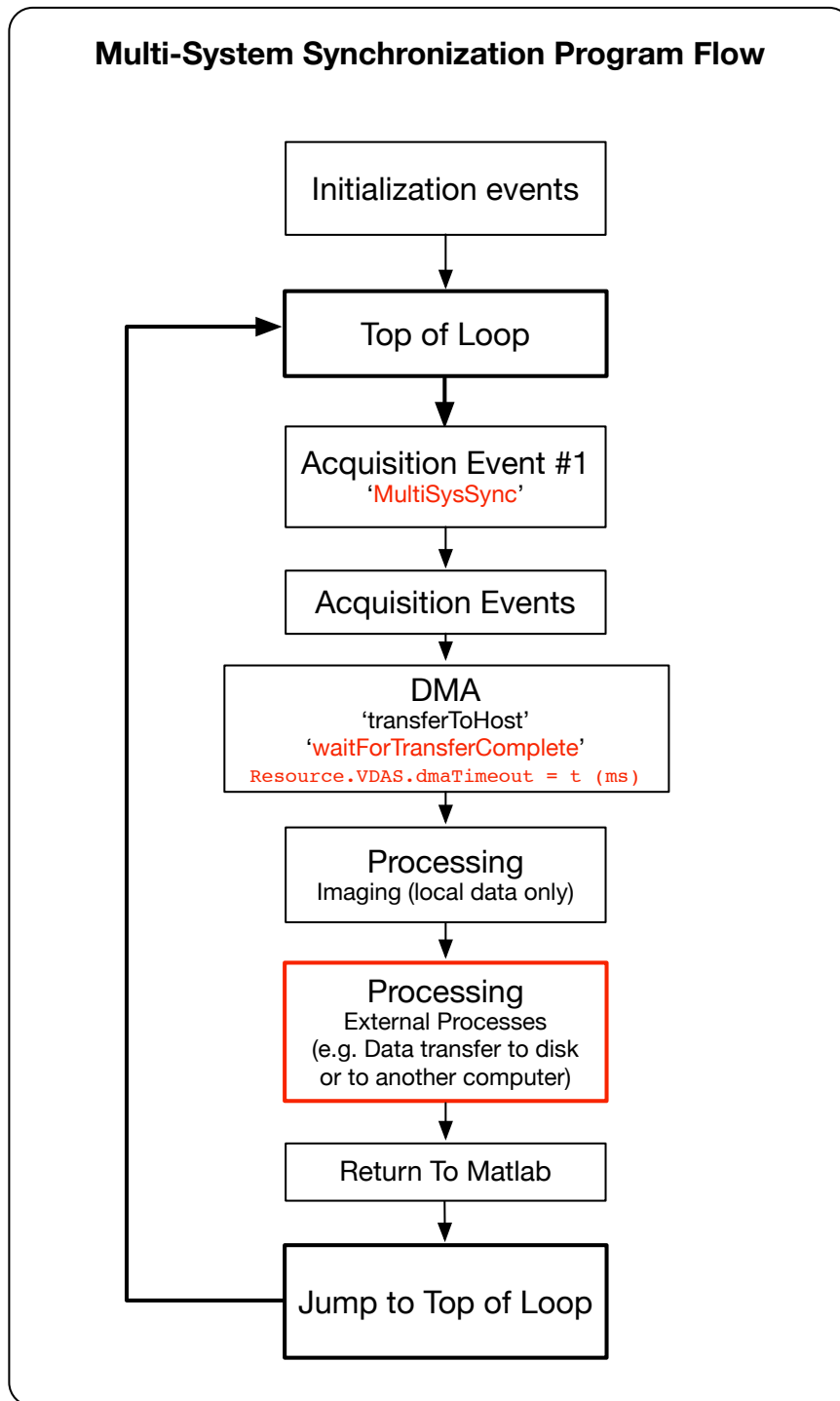


Figure 3. Multi-System Synchronization Program Flowchart. Red text and the red box highlight the important changes to include and consider.

6. Example Script

The following script is adapted from the basic single-angle plane wave imaging script to minimize complexity. Pay particular attention to the sections delineated with the comment lines:

```
% === add for 2-system sync ===  
... New synchronization code here ...  
% ===
```

The program is designed to be installed on both the Master and Slave (default) computers, but requires specifying which system is which using the variable “master”:

```
master = 0; % =1 Master =0 Slave Identify if this is the Master computer or the Slave
```

There are two differences between sequences, and both are related to timing. First, the DMA (data transfer to host) has a ‘waitForTransferComplete’ command to complement the ‘transferToHost’ command. This permits the user to detect when a software sequencer is waiting too long for a DMA operation to complete, usually because the acquisition referenced by the command hasn’t occurred. This is likely due, in turn, to a trigger input not received from the Sync box, indicating that some process has hung up the entire group of systems.

Second, the Master sets the frame rate using a Time to Next Acquisition Sequence Control (TTNA), while the Slaves should not: Slave sequencers should perform all operations as fast as they can in order to be “ready” before the Master completes its frame and issues the start trigger.

Key changes enabling Sync Module synchronization in the example script are documented in the code comments, and are listed here for convenience:

1. Set Master or Slave sequence using “master” variable
2. Set “Resource.VDAS.dmaTimeout” time (currently 10 seconds)
3. Add ‘MultiSysSync’ sequence control command to the first acquisition
4. Add ‘waitForTransferComplete’ sequence control to permit DMA timeout check
5. Add any extra external processing functions to transfer data among systems (section 8)

6.1 Additional Scripting Notes

- Keep in mind that the reported sequence frame rate is computed by dividing the elapsed time between Return to Matlab operations by the frameRateFactor variable. And an implicit return to matlab is inserted by VSX when the sequence jumps to event #1; the reported frame rate is then incorrect if explicit returnToMatlab commands are added to a sequence that jumps back to event #1. An initialization event before the top of the loop prevents the implicit return from being added.
- The trigger out delay argument is recognized in release 2.10, but was erroneously left undocumented in 2.10 and 2.11 releases. This mode is essential for assuring that a system can trigger itself using the external trigger cables, but is not used with the Sync module.
- The Process structure in the provided example script sets the compression factor to 1 to be able to see input noise on the display, without a transducer connected. This facilitates observing

whether the script is running. Normally, this parameter is set to 0.5 by default. This setting is only needed for initial verification of proper operation, and should be readjusted in the ultimate application script.

7. Verification and Phase Alignment

These instructions apply to any pair of Master and Slave systems. Of course, all slave systems should perform the same way.

1. Connect two oscilloscope probes to Master and Slave transducer connector outputs on the same connector pin (be sure to use an active channel!) and the ground clips to the shield spring.

It is often convenient to grasp a stiff inch-long wire conductor (e.g. a through-hole resistor leg) with the oscilloscope probe spring clip or “hat”, and hang it from the ZIF connector by inserting it into one pin socket. There will be no force other than slight friction holding it in place.

2. Trigger the scope using the MASTER signal. It is helpful to set the high voltage slider on both systems to 11.3 Volts, and the trigger level to around 1 Volt.

The value of 11.3 Volts is what one obtains by clicking in the slider field once.

3. Observe that the transmitter signals are not jittering with respect to each other. Measure the time delay between waveforms, if any. The SLAVE should be well within ± 2 ns of the MASTER (often any offset will be very difficult to measure).

The open circuit (no load) waveforms on the standard frequency system are “ugly”, but should look very similar to each other for identical TX settings. The high frequency model has a different transmitter, characterized by a much faster (less inductive) transition time, and so HF system waveforms look more like textbook tristate rectangular waveforms. The key is to observe phase jitter and phase shift between systems (should be very little).

Note that using trigger outputs to visualize event timing offset does not work. The Vantage trigger output is always synchronized to the system’s *internal* system clock, so the Slave trigger will not be synchronized to the Master trigger, even when it is using the Master’s system clock for acquisition timing. For this reason, *one cannot verify that the systems are operating synchronously by comparing the two output trigger signals* using an oscilloscope. One must verify synchronous operation by examining the transmitter outputs directly.

8. Real-time data transfer through TCP/IP

Before the start of this section, please make sure that the synchronization between multiple Vantage systems has been completed successfully by following sections 1-7. This section will focus on how to transfer data between systems in real-time using TCP/IP communication provided by the Matlab Instrument Control Toolbox. The example script will NOT work if the Instrument Control Toolbox is not installed. If the user does not have Instrument Control Toolbox, some substitutions are available in Mathworks file exchange website, like:

<https://www.mathworks.com/matlabcentral/fileexchange/24524-jtcp-actionstr-varargin->

<https://www.mathworks.com/matlabcentral/fileexchange/345-tcp-udp-ip-toolbox-2-0-6>

Please note that the user is responsible for the use of all source files downloaded from file exchange.

8.1 High Speed Ethernet Network Adapter and system configuration

A high-speed Ethernet network adapter is not required, but recommended. The script will also work with an Ethernet Switch, or direct connection between systems, as long as the IP address is set correctly. In our testing, two 10 Gigabit PCI-E Ethernet network adapters, D-Link DXE-810S, along with a 3M direct attach copper cable are dedicated to data transfer between two systems and an Ethernet Switch, NETGEAR ProSafe GS105, is used for four systems synchronization.

If DXE-810S is installed, the user should install the adapters and driver by following the manual in the CD-ROM provided in the product box. However, we recommend the user to skip the “Optimized Performance Tuning” in the manual. In our testing, it won’t increase the data transfer rate.

8.1.1 Set up IP address

The IP address should be defined for TCP/IP communication. Open Control Panel → Network and Internet → Network and Sharing Center, select "Change adapter settings" on the left panel. Double click the Ethernet card used for data transfer, select Properties, click TCP/IPv4 and then click "Properties". Set the IP address manually, like 192.168.0.1 for Master computer (192.168.0.2 to 192.168.0.4 for Slave computers). Leave Subnet mask as 255.255.255.0 and the default gateway is not required.

8.1.2 Set up a shared folder

Before running the example script, successfully creating a shared folder is the best way to evaluate the communication between host computers. The steps of creating a shared folder are summarized below:

1. Control Panel → System and Security → System. Select "Advanced system settings" on the left panel. (Shortcut: "WIN+R" and then enter "sysdm.cpl" in the command line)
2. On the "System Properties", under "Computer Name" tab, click "Change". Confirm that all computers are within the same workgroup. (The default one is "WORKGROUP" and it should work)
3. On the Master computer, right click the folder which will be shared and select "Properties".
4. Under "Sharing" tab, select "Advanced Sharing". Check "Share this folder" and allow full control of Everyone in the "Permissions".
5. Go to the slave computer, search for \\192.168.0.1. If the connection is successful, you will be asked to enter user name and password of the Master computer.
6. After successful connection, the shared folder should appear. Right click the folder and select "Map network drive". The shared folder is considered as a hardware drive on the slave computer.

8.2 Script Programming for real-time data transfer using TCP/IP

As described in section 6, the same program is designed to be installed on both the Master and Slave systems, but requires specifying which system is which using the variable “master”, as well as the correct IP address of the Master and Slave computer.

```
master = 0; % =1 Master =0 Slave Identify if this is the Master computer or the Slave
```

In order to successfully transfer large RF data, the user has to increase the default Java Heap Memory to avoid the following error: “There is not enough memory to creat the input buffer. Try specifying a smaller value for InputBuffer Size”. Please go to Matlab Preference → General → Java Heap Memory and change it to maximum to allow the maximum input buffer size.

Two examples are provided for 2 systems and 4 systems synchronization. The script for Slave systems are identical and the only difference is the TCP/IP objects defined in the Master system. Two sets of event sequence are used in the example script. First, real-time imaging without RF data transfer. In this set, only one frame is used and the user is able to control the UI controls of the Slave systems from the Master system, as shown in the video: <https://goo.gl/ox9mGE>. An external function is used to pass the command to the Slave systems. With this feature, the user is able to precisely control the parameters, including TGC, image depth range, PTool, and filter, among systems, instead of trying to set correct slider steps on each system.

Second, a “Transfer RF” push button is created in the Master UI only. After pressing the button, the event sequence jumps to the second set which acquires data with high frame rate (default timeToNextAcq: 50 us). After hundreds of acquisitions defined by numFramHFR, the RF data from the Slave systems will be transferred to the Master system and the combined data named “combinedRF” will be stored in the workspace. The event sequence will jump back to the first set after data combination. The detailed settings are described in SetUpL11_5vFlash_2SysSync_TCPIP.m or SetUpL11_5vFlash_4SysSync_TCPIP.m.

9. APPENDIX — Example script source code:

SetUpL11_5vFlash_MultiSysSync.m

```
% SetUpL11_5vFlash_MultiSysSync.m - Example of phase synchronizing two systems by sharing the 250 MHz clock and triggers
% Based on SetUpL11_5vFlash imaging with single plane wave transmit.
% Description:
% Sequence programming file for L11-5v Linear array, using a plane wave
% transmit and single receive acquisition. All 128 transmit and receive
% channels are active for each acquisition. Processing is asynchronous
% with respect to acquisition.
%
% The original script was modified to use in a multi-system shared clock configuration, in which an HDMI cable
% passes the 250 MHz clock from the "master" system Clock OUT to the synchronization box, which then redistributes
% that clock and some other synchronization signals to all of the systems. The SequenceControl command 'MultiSysSync'
% places each event sequence in a wait for trigger state; when the master system arrives at the MultiSysSync command
% it produces a trigger output that is sent to the sync box, which buffers and redistributes it
% to all systems (over the HDMI cables) as a trigger input. All systems, including the master, then continue executing
% their individual sequences at the same time. Here, the assumption is that all systems will run this same script,
% but only one script should set the variable 'master' to one, in line 37 below.
% See "Application Note Phase-Accurate Synchronization of Multiple Vantage Systems" for further documentation.
% Note the sections of code below identified by the comment: "% === add for multi-system sync ==="
%
% Last update:
% 7/31/2015 - Add synchronization logic for cable-only sync. Tested with Software 2.10.4.
% 9/8/2015 - Tested with 2.11.6
%
```

```
% Notice:
% This file is provided by Verasonics to end users as a programming
% example for the Verasonics Vantage Research Ultrasound System.
% Verasonics makes no claims as to the functionality or intended
% application of this program and the user assumes all responsibility
% for its use

clear all

filename = ('L11-5vFlash_MultiSysSync'); % assigning the matfile name to the variable 'filename' permits avoiding the VSX query

% === add for multi-system sync ===
master = 0; % =1 Master =0 Slave The Master script sets frame rate, and uses a TTNA command that is left out of the slave
scripts.
Resource.VDAS.dmaTimeout = 10*1000; % (ms) time software sequencer will wait for 'transferToHost'
% set this time long enough to permit launching the Slave sequence
% and then launching the Master sequence

% ===

% Define system parameters.
Resource.Parameters.numTransmit = 128; % number of transmit channels.
Resource.Parameters.numRcvChannels = 128; % number of receive channels.
Resource.Parameters.speedOfSound = 1540; % set speed of sound in m/sec before calling computeTrans
Resource.Parameters.simulateMode = 0;
% Resource.Parameters.simulateMode = 1 forces simulate mode, even if hardware is present.
% Resource.Parameters.simulateMode = 2 stops sequence and processes RcvData continuously.

% Specify Trans structure array.
Trans.name = 'L11-5v';
Trans.units = 'mm'; % Explicit declaration avoids warning message when selected by default
Trans = computeTrans(Trans); % L11-5v transducer is 'known' transducer so we can use computeTrans.
Trans.maxHighVoltage = 50; % set maximum high voltage limit for pulser supply.

% Specify SFormat structure array.
SFormat.transducer = 'L11-5v'; % 128 element linear array
SFormat.scanFormat = 'RLIN'; % rectangular linear array scan
SFormat.radius = 0; % ROC for curved lin. or dist. to virt. apex
SFormat.theta = 0;
SFormat.numRays = 1; % no. of Rays (1 for Flat Focus)
SFormat.FirstRayLoc = [0,0,0]; % x,y,z
SFormat.rayDelta = 128*Trans.spacing; % spacing in radians(sector) or dist. between rays (wvlnghts)
SFormat.startDepth = 5; % Acquisition depth in wavelengths
SFormat.endDepth = 192; % This should preferably be a multiple of 128 samples.

% Specify PData structure array.
PData.sFormat = 1; % use first SFormat structure.
PData.pdeltaX = Trans.spacing;
PData.pdeltaZ = 0.5;
PData.Size(1) = ceil((SFormat.endDepth-SFormat.startDepth)/PData.pdeltaZ); % startDepth, endDepth and pdelta set PData.Size.
PData.Size(2) = ceil((Trans.numElements*Trans.spacing)/PData.pdeltaX);
PData.Size(3) = 1; % single image page
PData.Origin = [-Trans.spacing*(Trans.numElements-1)/2,0,SFormat.startDepth]; % x,y,z of upper lft crnr.

% Specify Media object. 'ptl.m' script defines array of point targets.
ptl;
Media.function = 'movePoints';

% Specify Resources.
Resource.RcvBuffer(1).datatype = 'int16';
Resource.RcvBuffer(1).rowsPerFrame = 4096; % this size allows for maximum range
Resource.RcvBuffer(1).colsPerFrame = Resource.Parameters.numRcvChannels;
Resource.RcvBuffer(1).numFrames = 100; % 100 frames used for RF cineloop.
Resource.InterBuffer(1).datatype = 'complex';
Resource.InterBuffer(1).rowsPerFrame = 1024; % this is for greatest depth
Resource.InterBuffer(1).colsPerFrame = PData.Size(2);
Resource.InterBuffer(1).numFrames = 1; % one intermediate buffer needed.
Resource.ImageBuffer(1).datatype = 'double';
Resource.ImageBuffer(1).rowsPerFrame = 1024;
Resource.ImageBuffer(1).colsPerFrame = PData.Size(2);
Resource.ImageBuffer(1).numFrames = 10;
Resource.DisplayWindow(1).Title = filename;
Resource.DisplayWindow(1).pdelta = 0.35;
ScrnSize = get(0,'ScreenSize');
DwWidth = ceil(PData.Size(2)*PData.pdeltaX/Resource.DisplayWindow(1).pdelta);
DwHeight = ceil(PData.Size(1)*PData.pdeltaZ/Resource.DisplayWindow(1).pdelta);
Resource.DisplayWindow(1).Position = [250,(ScrnSize(4)-(DwHeight+150))/2, ... % lower left corner position
DwWidth, DwHeight];
Resource.DisplayWindow(1).ReferencePt = [PData.Origin(1),PData.Origin(3)]; % 2D imaging is in the X,Z plane
Resource.DisplayWindow(1).Colormap = gray(256);
```



```
% Specify Transmit waveform structure.
TW.type = 'parametric';
TW.Parameters = [Trans.frequency, .67, 2, 1]; % A, B, C, D

% Specify TX structure array.
TX(1).waveform = 1; % use 1st TW structure.
TX(1).Origin = [0.0,0.0,0.0]; % flash transmit origin at (0,0,0).
TX(1).focus = 0;
TX(1).Steer = [0.0,0.0]; % theta, alpha = 0.
TX(1).Apod = ones(1,Trans.numelements);
TX(1).Delay = computeTXDelays(TX(1));

% Specify TGC Waveform structure.
TGC.CntrlPts = [0,200,344,452,606,747,870,920];
TGC.rangeMax = SFormat.endDepth;
TGC.Waveform = computeTGCWaveform(TGC);

% Specify Receive structure arrays -
% endDepth - add additional acquisition depth to account for some channels
% having longer path lengths.
% InputFilter - The same coefficients are used for all channels. The
% coefficients below give a broad bandwidth bandpass filter.
maxAcqLength = sqrt(SFormat.endDepth^2 + (Trans.numelements*Trans.spacing)^2) - SFormat.startDepth;
wlsPer128 = 128/(4*2); % wavelengths in 128 samples for 4 samplesPerWave
Receive = repmat(struct('Apod', ones(1,Trans.numelements), ...
    'startDepth', SFormat.startDepth, ...
    'endDepth', SFormat.startDepth + wlsPer128*ceil(maxAcqLength/wlsPer128), ...
    'TGC', 1, ...
    'bufnum', 1, ...
    'framenum', 1, ...
    'acqNum', 1, ...
    'samplesPerWave', 4, ...
    'mode', 0, ...
    'callMediaFunc', 1),1,1,Resource.RcvBuffer(1).numFrames);

% - Set event specific Receive attributes.
for i = 1:Resource.RcvBuffer(1).numFrames
    % -- Acquisition for full frame.
    Receive(i).framenum = i;
end

% Specify Recon structure arrays.
Recon = struct('senscutoff', 0.6, ...
    'pdatanum', 1, ...
    'rcvBufFrame', -1, ... % use most recently transferred frame
    'IntBufDest', [1,1], ...
    'ImgBufDest', [1,-1], ... % auto-increment ImageBuffer each recon
    'RINums', 1, 1);

% Define ReconInfo structures.
ReconInfo = repmat(struct('mode', 0, ...
    'txnum', 1, ...
    'rcvnum', 1, ...
    'regionnum', 0), 1, 1);

% Specify Process structure array.
pers = 20;
Process(1).classname = 'Image';
Process(1).method = 'imageDisplay';
Process(1).Parameters = {'imgbufnum',1,... % number of buffer to process.
    'framenum',-1,... % (-1 => lastFrame)
    'pdatanum',1,... % number of PData structure to use
    'norm',1,... % normalization method(1 means fixed)
    'pgain',3.0,... % pgain is image processing gain
    'persistMethod','simple',...
    'persistLevel',pers,...
    'interp',1,... % method of interpolation (1=4pt interp)
    'compression',1,... % == set to 1 for use without transducer (normally 0.5)
    'reject',5,... % reject level
    'mappingMode','full',...
    'display',1,... % display image after processing
    'displayWindow',1};

% Specify SeqControl structure arrays.
nsc = 1; % nsc-1 is count of SeqControl objects
SeqControl(nsc).command = 'jump'; % jump back to event in argument set below
JUMP = nsc; nsc = nsc+1; % set a mnemonic for the jump command index

SeqControl(nsc).command = 'timeToNextAcq'; % time between frames
SeqControl(nsc).argument = 20*1000; % 20 msec for 50 fps
TTNA = nsc; nsc = nsc+1;
```



```
SeqControl(nsc).command = 'returnToMatlab';
RTNMatlab = nsc;    nsc = nsc+1;

SeqControl(nsc).command = 'noop';
SeqControl(nsc).argument = 10*5; % 1 microsec for 5 counts (200 ns per count)
NOOP = nsc;    nsc = nsc+1;

% === add for multi-system sync ===
SeqControl(nsc).command = 'multiSysSync';
SeqControl(nsc).condition = 'normal'; % 'normal' uses internal trigger; other options are 'BNC_Rising' and 'BNC_Falling'
MSSYNC= nsc;    nsc = nsc+1;
% ===

% SeqControl(nsc).command = 'sync'; % synchronize SW and HW sequencers (optional)
% SYNC = nsc;
% nsc = nsc+1;

n = 1; % n is count of Events

Event(n).info = 'NOOP'; % initial dummy event to prevent automatic insertion of Rtn2Matlab when jumping to event #1
Event(n).tx = 0;
Event(n).rcv = 0;
Event(n).recon = 0;
Event(n).process = 0;
Event(n).seqControl = NOOP;
n=n+1;

SeqControl(JUMP).argument = n; % set jump address to back to top of loop

% Acquire all frames defined in RcvBuffer
for i = 1:Resource.RcvBuffer(1).numFrames

    Event(n).info = 'Full aperture acquisition';
    Event(n).tx = 1; % use 1st TX structure.
    Event(n).rcv = i; % use ith Rcv structure for the ith frame.
    Event(n).recon = 0; % no reconstruction.
    Event(n).process = 0; % no processing
    % === add for multi-system sync ===
    if master
        Event(n).seqControl = [TTNA, MSSYNC]; % only add 'MultiSysSync' to the first acquisition event in a frame
    else
        Event(n).seqControl = MSSYNC; % === No TTNA for the Slave system, so that it runs faster than the Master sequence
    end
    % ===
    n = n+1;

    Event(n).info = 'DMA';
    Event(n).tx = 0; % use 1st TX structure.
    Event(n).rcv = 0; % use ith Rcv structure for the ith frame.
    Event(n).recon = 0; % no reconstruction.
    Event(n).process = 0; % no processing
    Event(n).seqControl = [nsc,nsc+1]; % SeqControl struct defined below.
    SeqControl(nsc).command = 'transferToHost';
    % === add for multi-system sync ===
    SeqControl(nsc+1).command = 'waitForTransferComplete'; % used to inform the user that acquisition hasn't occurred,
    % presumably because a trigger was not received within DMA

    timeout
        SeqControl(nsc+1).argument = nsc;
        nsc = nsc + 2;
        % ===
        n = n+1;

    Event(n).info = 'Reconstruct & Process';
    Event(n).tx = 0; % no transmit
    Event(n).rcv = 0; % no rcv
    Event(n).recon = 1; % reconstruction
    Event(n).process = 1; % processing
    if floor(i/5) == i/5 % Exit to Matlab every 5th frame
        Event(n).seqControl = RTNMatlab; % return to Matlab
    else
        Event(n).seqControl = 0;
    end
    n = n+1;

end

Event(n).info = 'Jump back';
Event(n).tx = 0; % no TX
```

```

Event(n).rcv = 0;          % no Rcv
Event(n).recon = 0;       % no Recon
Event(n).process = 0;
Event(n).seqControl = JUMP; % jump command (must have explicit 'returnToMatlab' in sequence if not to event #1)

% User specified UI Control Elements
% - Sensitivity Cutoff
UI(1).Control = {'UserB7','Style','VsSlider','Label','Sens. Cutoff',...
    'SliderMinMaxVal',[0,1.0,Recon(1).senscutoff],...
    'SliderStep',[0.025,0.1],'ValueFormat','%1.3f'};
UI(1).Callback = text2cell('%-UI#1Callback');

% - Range Change
UI(2).Control = {'UserA1','Style','VsSlider','Label','Range',...
    'SliderMinMaxVal',[64,320,SFormat.endDepth],'SliderStep',[0.1,0.2],'ValueFormat','%3.0f'};
UI(2).Callback = text2cell('%-UI#2Callback');

% Specify factor for converting sequenceRate to frameRate.
frameRateFactor = 5;

% Save all the structures to a .mat file.
save(filename);
    VSX % setting 'filename' variable permits running VSX without query
return

% **** Callback routines to be converted by text2cell function. ****
%-UI#1Callback - Sensitivity cutoff change
ReconL = evalin('base','Recon');
for i = 1:size(ReconL,2)
    ReconL(i).senscutoff = UIValue;
end
assignin('base','Recon',ReconL);
Control = evalin('base','Control');
Control.Command = 'update&Run';
Control.Parameters = {'Recon'};
assignin('base','Control',Control);
return
%-UI#1Callback

%-UI#2Callback - Range change
simMode = evalin('base','Resource.Parameters.simulateMode');
% No range change if in simulate mode 2.
if simMode == 2
    set(hObject,'Value',evalin('base','SFormat.endDepth'));
    return
end
range = UIValue;
assignin('base','range',range);
SFormat = evalin('base','SFormat');
SFormat.endDepth = range;
assignin('base','SFormat',SFormat);
evalin('base','PData.Size(1) = ceil((SFormat.endDepth-SFormat.startDepth)/PData.pdeltaZ);');
evalin('base','[PData.Region,PData.numRegions] = createRegions(PData);');
evalin('base','Resource.DisplayWindow(1).Position(4) = ceil(PData.Size(1)*PData.pdeltaZ/Resource.DisplayWindow(1).pdelta);');
Receive = evalin('base','Receive');
Trans = evalin('base','Trans');
maxAcqLength = sqrt(range^2 + (Trans.numElements*Trans.spacing)^2)-SFormat.startDepth;
wlsPer128 = 128/(4*2);
for i = 1:size(Receive,2)
    Receive(i).endDepth = SFormat.startDepth + wlsPer128*ceil(maxAcqLength/wlsPer128);
end
assignin('base','Receive',Receive);
evalin('base','TGC.rangeMax = SFormat.endDepth;');
evalin('base','TGC.Waveform = computeTGCWaveform(TGC);');
Control = evalin('base','Control');
Control.Command = 'update&Run';
Control.Parameters = {'SFormat','PData','Receive','Recon','DisplayWindow','ImageBuffer'};
assignin('base','Control',Control);
assignin('base','action','displayChange');
return

```

*This document is intended for use by Verasonics customers only.
Do not duplicate or distribute without permission.*

Verasonics, Inc.
11335 NE 122nd Way, Suite 100
Kirkland, WA 98034
USA